

Enseignants : ... S. NOUH &A. ZAKRANI

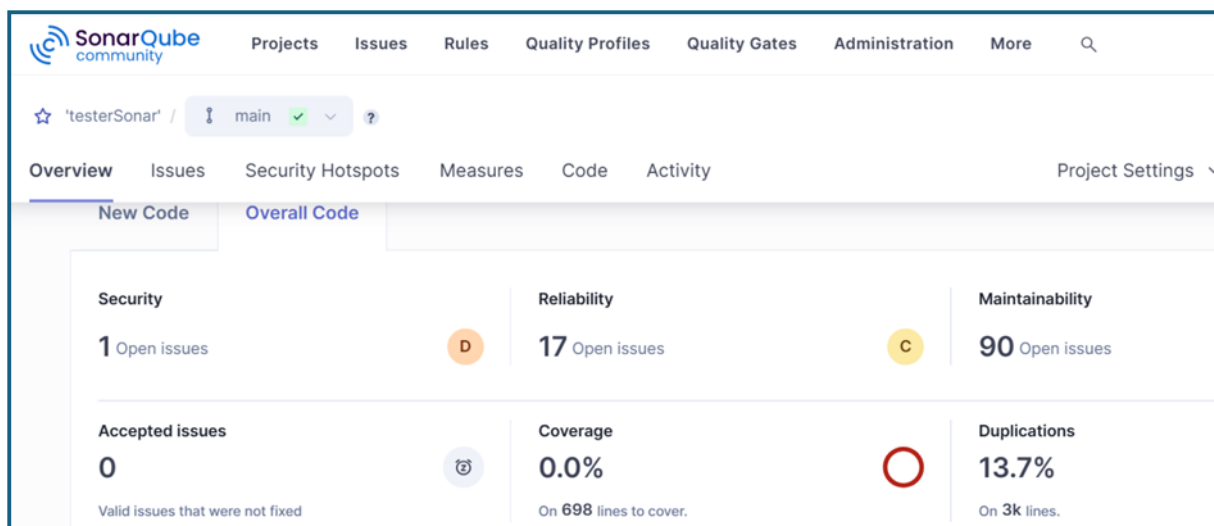
NOTE :

Veillez répondre sur la feuille de sujet fournie.

Veillez mentionner votre nom et prénom dans la place correspondante ci-dessus.

Pour la partie du QCM, veuillez sélectionner la ou les bonnes réponses et mettez une croix (X) dans la case correspondante du tableau des réponses dans la page 11

Exercice 1 : Une analyse d'un petit projet avec SonarQube a fourni les résultats suivants :



Question : À partir des résultats de l'analyse SonarQube, expliquez brièvement la signification des indicateurs obtenus, évaluez la qualité du code et proposez des améliorations pour l'optimiser.

[illegible]

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

QCM : Sélectionner la ou les bonnes réponses et mettez une croix (X) dans la case correspondante du tableau des réponse dans la page 11

1. **Le modèle de qualité logicielle ISO 25010 définit :**
 - a) Uniquement la qualité d'utilisation
 - b) Uniquement la qualité du produit
 - c) La qualité d'utilisation et la qualité du produit
 - d) La gestion des coûts
2. **La portabilité d'un logiciel désigne :**
 - a) Sa capacité à s'exécuter sur différents environnements
 - b) Son aptitude à détecter les erreurs
 - c) Son respect des normes ISO
 - d) Sa capacité à minimiser les ressources mémoire et processeur utilisées
3. **Le principe de responsabilité unique (Single Responsibility Principle - SRP) signifie :**
 - a) Une classe ne doit avoir qu'une seule raison de changer
 - b) Une classe peut avoir plusieurs sous-classes
 - c) Un module peut changer ses responsabilités selon l'environnement
 - d) Les classes doivent implémenter des interfaces
4. **Selon le principe Open/Close, un module logiciel doit être :**
 - a) Ouvert à la modification et fermé à l'extension
 - b) Ouvert à l'extension et fermé à la modification
 - c) Toujours modifiable et extensible sans restriction
 - d) Fermé à toute évolution
5. **Quel est l'objectif du principe de Substitution de Liskov ?**
 - a) Une instance d'une classe doit pouvoir être substituée sans modification par une instance d'une sous-classe.
 - b) Empêcher la réutilisation des classes
 - c) Modifier le comportement des classes enfants indépendamment des parents
 - d) Interdire l'héritage des classes
6. **Le principe d'inversion des dépendances (Dependency Inversion Principle - DIP) permet :**
 - a) De faire dépendre les modules aux détails
 - b) De coupler fortement les composants d'un logiciel
 - c) De découpler les composants en s'appuyant sur des abstractions
 - d) De forcer l'utilisation des classes concrètes au lieu des classes abstraites
7. **Quels sont les objectifs des tests fonctionnels ?**
 - a) Vérifier la conformité d'un logiciel aux exigences métier
 - b) Tester les performances matérielles
 - c) S'assurer de la validité du code source
 - d) Détecter les failles de cybersécurité
8. **Quel framework est souvent utilisé pour l'automatisation des tests fonctionnels ?**
 - a) Selenium
 - b) Mockito
 - c) JUnit
 - d) JMeter
 - L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
 - Aucun document **n'est autorisé.**
 - Calculatrice **non autorisée.**

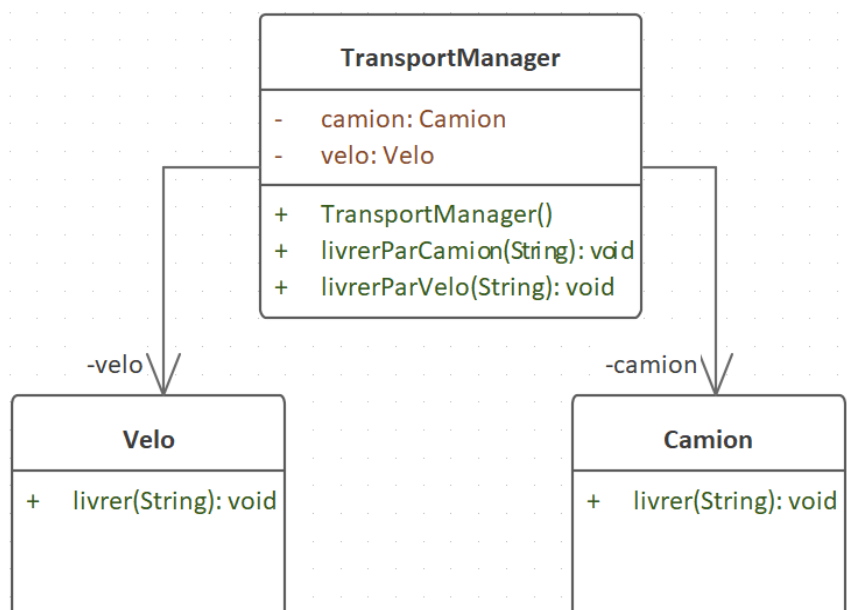
9. Quelle condition est nécessaire pour respecter le principe de substitution de Liskov ?
 - a) La sous-classe doit être une classe fille de la superclasse sans redéfinir les méthodes héritées.
 - b) La sous-classe doit hériter toutes les propriétés de la superclasse.
 - c) Les méthodes de la sous-classe doivent être compatibles avec celles de la superclasse sans affaiblir ou renforcer des conditions
 - d) La sous-classe doit avoir une et une seule raison pour être modifiée.
10. Quelle commande permet de charger une page web par son URL en utilisant SeleniumWebDriver (avec `driver = new ChromeDriver();`) ?
 - a) `driver.findElement(By.url())`
 - b) `driver.get(String url)`
 - c) `driver.navigate().to(String url)`
 - d) `driver.load(String url)`
11. Soit la commande suivante :


```
mvn clean verify sonar:sonar -Dsonar.projectKey=MonProjetPFE -
Dsonar.projectName='MonProjetPFE' -Dsonar.host.url=http://localhost:9000 -
Dsonar.token=sqp_g04e23929cf756105973fd5546218be4d6b3c8fc
```

 Quel est le rôle de la partie `sonar:sonar` dans cette commande ?
 - a) Elle compile le code source en vue d'un scan SonarQube.
 - b) Elle lance le processus d'analyse SonarQube pour envoyer les résultats du projet vers un serveur SonarQube.
 - c) Elle déploie le projet sur un serveur SonarQube.
 - d) Elle effectue une analyse de la couverture de code pour les tests unitaires.
12. Quel langage de programmation SonarQube peut-il analyser ?
 - a) Java
 - b) Python
 - c) C++
 - d) Tous les langages mentionnés ci-dessus

Exercice 2 : Principes SOLID : Une entreprise de transport souhaite développer une application qui gère différents types de véhicules utilisés pour effectuer des livraisons. Actuellement, la classe `TransportManager` est directement couplée à des implémentations spécifiques des véhicules (`Camion` et `Vélo`). Cependant, l'entreprise prévoit d'intégrer d'autres moyens de transport à l'avenir, comme des drones ou des bateaux, sans modifier la classe principale.

Voici le code existant de l'application :



- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

```
class TransportManager {
    private Camion camion;
    private Velo velo;

    public TransportManager() {this.camion = new Camion();
                                this.velo = new Velo(); }
    public void livrerParCamion(String destination) {
                                camion.livrer(destination); }
    public void livrerParVelo(String destination) {
                                velo.livrer(destination); }
}

class Camion {
    public void livrer(String destination) {
        System.out.println("Livraison par camion vers " + destination); }
}

class Velo {
    public void livrer(String destination) {
        System.out.println("Livraison par vélo vers " + destination); }
}
```

Travail à faire :

- 1) Est-ce que le diagramme de classes et le code ci-dessus respectent le principe de l'Inversion de Dépendance (DIP), justifier votre réponse ?

.....

.....

.....

- 2) Si le diagramme de classes et le code ci-dessus ne respectent pas ce principe, proposer une autre version (donner juste diagramme de classes) qui respecte ce principe DIP et expliquer pourquoi votre nouvelle version respecte ce principe.

[illegible]

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

Exercice 3 : Test unitaire avec JUnit Jupiter (NB : il y a un annexe à la page 12 qui peut vous aider)

Voici une implémentation de la classe qui calcule le nombre d'occurrences d'une lettre dans une chaîne de caractères, ainsi qu'une classe de test unitaire utilisant **JUnit Jupiter**.

```
public class OccurrenceCounter {  
    public int compterOccurrences(String str, char letter) {  
        if (str == null) {  
            throw new IllegalArgumentException("La chaîne de caractères ne peut pas être null.");  
        }  
        int count = 0;  
        for (char c : str.toCharArray()) {  
            if (c == letter) { count++; }  
        }  
        return count;  
    }  
}
```

Questions :

1. Compléter le code en écrivant un cas de test qui vérifie le cas où la lettre recherchée est absente de la chaîne de caractères. Ajouter un nom clair et explicite à ce cas de test à l'aide de l'annotation `@DisplayName`. **(Compléter la partie commentée par //Question 1)**
2. Compléter le code en écrivant un cas de test qui vérifie que la méthode lève une exception appropriée lorsqu'une chaîne de caractères **null** est fournie en entrée. **(Compléter la partie commentée par //Question 2)**
3. Nous disposons d'un fichier CSV nommé **occurrence_test_data.csv** dans le répertoire `src/test/resources` contenant plusieurs données de test. Compléter le code en écrivant un test paramétré en utilisant le fichier CSV donné. **(Compléter la partie commentée par //Question 3)**

```
str,letter,expected  
"programmation",a,2  
"programmation",z,0  
"",a,0  
"Programmation",p,0  
"aaaa",a,4  
"banana",a,3
```

```
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.*;  
class OccurrenceCounterTest {  
    @Test  
    void testCompterOccurrencesLettrePresente() {  
        OccurrenceCounter counter = new OccurrenceCounter();  
        int result = counter.compterOccurrences("programmation", 'a');  
        assertEquals(2, result, "La lettre 'a' devrait apparaître 2 fois."); }  
}
```

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

// Question 1

```

@.....
@.....
void testCompterOccurrencesLettreAbsente() {
  OccurrenceCounter counter = new OccurrenceCounter();
  .....
  .....}
  
```

// Question 2

```

@.....
void testCompterOccurrencesNullInput() {
  OccurrenceCounter counter = new OccurrenceCounter();
  .....
  ..... }
  
```

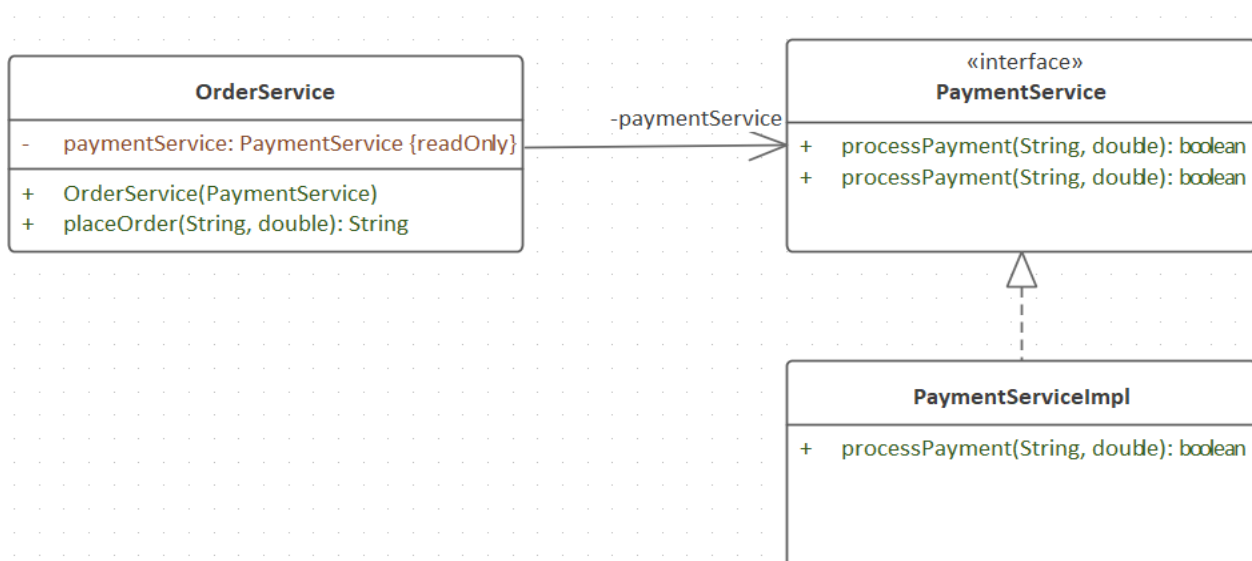
// Question 3

```

@.....
@DisplayName("Test paramétré pour plusieurs cas d'utilisation")
@.....
void testCompterOccurrencesParametre(String str, char letter, int expected) {
  OccurrenceCounter counter = new OccurrenceCounter();
  int result = counter.compterOccurrences(str, letter);
  assertEquals(expected, result, "Le nombre d'occurrences est incorrect."); } }
  
```

Exercice 4 : Framework Mockito : (NB : il y a un annexe à la page 12 qui peut vous aider)

Une entreprise développe une application de gestion de commandes. La classe `OrderService` dépend d'un service externe `PaymentService` pour effectuer les paiements. Nous voulons tester le comportement de `OrderService` sans appeler réellement le service de paiement.



- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

```
public interface PaymentService {
    boolean processPayment(String orderId, double amount);
}

public class PaymentServiceImpl implements PaymentService {
    @Override
    public boolean processPayment(String orderId, double amount) {
        // Simulation d'une logique de paiement externe
        System.out.println("Paiement en cours pour la commande: " + orderId);
        return amount > 0; // Le paiement réussit si le montant est positif
    }
}

public class OrderService {
    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public String placeOrder(String orderId, double amount) {
        if (paymentService.processPayment(orderId, amount)) {
            return "Commande confirmée";
        } else { return "Échec du paiement"; }
    }
}
```

Questions :

- 1) En analysant le diagramme de classes ci-dessus, est-ce qu'il respecte le principe de « **Ouvert à l'extension et fermé à la modification** » ? justifier votre réponse ?

.....

.....

.....

.....

.....

- 2) Dans le programme java ci-dessous nous avons implémenté un cas de test en utilisant le framework Mockito.

Travail à faire : Compléter les (-----) par les instructions qui manquent.

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

```
import org.junit.jupiter.api.Test;
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;

class OrderServiceTest {

    @Test
    void testPlaceOrder_PaymentSuccessful() {
        // Création d'un mock de PaymentService basé sur l'interface
        PaymentService mockPaymentService = -----;

        /* Définition du comportement du mock : paiement réussi (orderId="123", amount=100.0 et
           expected =true) */
        -----;

        OrderService orderService = new OrderService(mockPaymentService);
        String result = orderService.placeOrder("123", 100.0);

        assertEquals("Commande confirmée", result, "Le paiement devrait réussir.");

        // Vérification que la méthode a été appelée exactement une fois avec les bons paramètres
        -----;
    }
}
```

Exercice 5 : Automatisation du test fonctionnel avec Selenium :

(NB : il y a un annexe à la page 12 qui peut vous aider)

Nous souhaitons tester une partie d'une application web qui permet de résoudre une équation de deuxième degré de la forme : $ax^2+bx+c=0$ avec a, b et c sont des paramètres réels à saisir dans le formulaire de la page web ci-dessous. Une partie du code source html de cette page est donnée ci-dessous.

Remarque : la seule partie du code que vous devez utiliser dans les réponses est la partie `<div...>.....jusqu'à </div>`. La partie `<script>.....</script>` vous pouvez l'ignorer complètement)

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

Résolution d'une équation de second degré de la forme: $a \cdot X^2 + b \cdot X + c = 0$ a: 1 b: 2 c: 1 Résoudre Résultat : Une solution unique : x = -1.00	<pre> <body> <div class="container"> <h2>Résolution d'une équation de second degré la forme: a*X<sup>2</sup>+b*X+c=0 </h2> <label for="a"><h2>a: </h2></label> <input type="number" id="a" required>
 <label for="b"><h2>b: </h2></label> <input type="number" id="b" required>
 <label for="c"><h2>c: </h2></label> <input type="number" id="c" required>
 <button id="trouveSolutions" onclick="resoudreEquation()"> <h2>Résoudre</h2></button> <h3>Résultat :</h3> <label id="resultat" name="resultat"></label> </div> <script> function resoudreEquation() { let a = parseFloat(document.getElementById('a').value); let b = parseFloat(document.getElementById('b').value); let c = parseFloat(document.getElementById('c').value); let resultat = document.getElementById('resultat'); if (isNaN(a) isNaN(b) isNaN(c)) { resultat.innerHTML = "Veuillez entrer des valeurs valides."; return; } if (a === 0) { resultat.innerHTML = "Ce n'est pas une équation du second degré."; return; } let delta = b * b - 4 * a * c; if (delta > 0) { let x1 = (-b + Math.sqrt(delta)) / (2 * a); let x2 = (-b - Math.sqrt(delta)) / (2 * a); resultat.innerHTML = `Deux solutions : x1 = \${x1.toFixed(2)}, x2 = \${x2.toFixed(2)} `; } else if (delta === 0) { let x = -b / (2 * a); resultat.innerHTML = `Une solution unique : x = \${x.toFixed(2)} `; } else {resultat.innerHTML = "Aucune solution réelle (delta négatif)."; } } </script> </body> </pre>
---	--

Figure 1 :page web de résolution d'une équation de deuxième degré avec exemple d'exécution.

En utilisant SeleniumWebDriver, nous voulons écrire les deux cas de test suivants en utilisant le navigateur google chrome un cas de test **testEquationAvecUneSeuleSolution()** qui permet de vérifier que l'application fonctionne bien dans le cas d'une équation dont $\Delta = b^2 - 4 \cdot a \cdot c$ est égal à 0. Pour ce faire, on commence par identifier les composants html qui correspondent aux différents coefficients de l'équation a, b, et C et y saisir les valeurs : 1,2,1 dans cet ordre (c-à-dire a=1, b=2 et c=1). Ensuite on identifie le composant html qui correspond au bouton « Résoudre » et on fait un clic sur ce bouton et finalement on identifie le composant html qui correspond au label qui affiche le résultat et on récupère le texte qu'il contient et on le compare avec la chaîne de caractères:"Une solution unique : x = -1.00 ".

Travail à faire : compléter le code suivant qui implémente ces deux cas de test.

```

package org.example;
import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.BeforeAll;

```

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

```
import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import static org.junit.jupiter.api.Assertions.assertEquals;
public class testPageEquation {
    private static WebDriver driver;

    @BeforeAll
    public static void setUpAll() {driver = new ChromeDriver();}
    @AfterAll
    public static void tearDown() {driver.quit();}

    @Test
    public void testEquationAvecUneSeuleSolution() throws InterruptedException {
        String baseUrl = "www.selenium2025.com";
        driver. ....(baseUrl);
        .....coeff_a = driver. ....(By.id("a"));
        coeff_a. .... ("1");
        .....coeff_b = driver. .... (By.id("b"));
        coeff_b. .... ("2");
        .....coeff_c = driver. .... (By.id("c"));
        coeff_c. .... ("1");
        .....bouton_v = driver. ....(By.id("trouveSolutions"));
        bouton_v. ....;
        WebElement res = .....;
        String contenuRes=res. ....;
        assertEquals(contenuRes,"Une solution unique : x = -1.00");    }
    }
}
```

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

Fiche de réponse pour QCM

Veuillez mettre X sur la bonne réponse 

Question	A	B	C	D
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**

Annexe

JUnit	Mockito
▪ public static void assertEquals(Object expected, Object actual) ▪ public static void assertTrue(boolean condition) ▪ public static void assertFalse(boolean condition) ▪ public static void assertNull(Object actual) ▪ public static void assertNotNull(Object actual) ▪ public static <T extends Throwable> T assertThrows(Class<T> exceptionType, Executable executable) ▪ @ParameterizedTest ▪ @BeforeEach ▪ @AfterEach ▪ @BeforeAll ▪ @AfterAll	▪ mock(): permet de créer un objet fictif ▪ when(): permet de définir le comportement d'un objet fictif lors d'un appel de méthode ▪ thenReturn(): permet de définir la valeur de retour d'un appel de méthode fictif ▪ verify(): permet de vérifier que les méthodes ont été appelées avec les arguments appropriés ▪ any(): permet de spécifier un argument quelconque pour une vérification de méthode ▪ doReturn(), doThrow(), doAnswer(), etc.: permet de définir un comportement personnalisé pour un appel de méthode fictif ▪ reset(): permet de réinitialiser un objet fictif pour une utilisation ultérieure.

SeleniumWebDriver
▪ WebDriver driver = new ChromeDriver(): permet de démarrer un nouveau navigateur Chrome ▪ driver.get("url"): permet de charger une page Web spécifique ▪ driver.findElement(By.xpath("xpath")): permet de trouver un élément sur la page en utilisant un chemin d'accès XPath ▪ driver.click(): permet de cliquer sur un élément ▪ driver.sendKeys("input"): permet de saisir du texte dans un champ de formulaire ▪ driver.getTitle(): permet de récupérer le titre de la page actuelle ▪ driver.quit(): permet de fermer le navigateur et de terminer la session de test ▪ driver.close(): permet de fermer la fenêtre du navigateur actuelle. ▪ getText() : permet de récupérer le texte d'un élément web ▪ driver.findElement(By.id("id")) ▪ driver.findElement(By.className("class")) ▪ driver.findElement(By.name("name")) ▪ driver.findElement(By.tagName("tag")) ▪ driver.findElement(By.linkText("text")) ▪ driver.findElement(By.partialLinkText("text")) ▪ driver.findElement(By.cssSelector("selector"))

- L'usage des téléphones portables et de tout objet connecté (montre intelligente, etc.) **est strictement interdit.**
- Aucun document **n'est autorisé.**
- Calculatrice **non autorisée.**